DIPLOMATIC FLEET

Manual técnico del sistema

Primera entrega de la plataforma de gestión vehicular institucional

Daniela Largo Corredor

Xiomara Lobatón Galindo

Bryan Steven Pardo Núñez

María Camila Pacheco Morales

Politécnico Grancolombiano

Práctica Aplicada a Sistemas

EDILBERTO TORRES ORTIZ

Bogotá, Colombia

Octubre de 2026

Resumen

Este manual técnico documenta la instalación, configuración, arquitectura, persistencia, seguridad, integración, pruebas, mantenimiento y solución de problemas de la primera entrega de Diplomatic Fleet. El sistema utiliza Vue.js y TypeScript en el frontend, Kotlin y Spring Boot en el backend, Spring Data JPA para persistencia, Flyway para migraciones y PostgreSQL como base de datos. Los módulos de Clientes institucionales y Vehículos operan de extremo a extremo; los demás módulos se encuentran como vistas navegables preparadas para su integración. El documento diferencia configuraciones temporales de demostración y requisitos de producción, e incluye al final los requerimientos mínimos de hardware, software, red, puertos y ejecución.

Palabras clave: manual técnico, Kotlin, Spring Boot, Vue.js, PostgreSQL, Flyway, API REST, arquitectura por capas.

Tabla de contenido

1. Propósito y alcance técnico 5

2. Descripción general del sistema 5

3. Arquitectura de solución 6

4. Estructura del código fuente 11

5. Tecnologías y dependencias 12

6. Configuración del backend 13

7. Configuración del frontend 14

8. Base de datos y migraciones 15

9. Módulo de clientes institucionales 16

10. Módulo de vehículos 17

11. Seguridad y autenticación temporal 17

12. Manejo de errores y validaciones 18

13. Instalación desde cero 19

14. Ejecución y apagado controlado 20

15. Pruebas y aseguramiento de calidad 20

16. Control de versiones y despliegue 21

17. Copia, migración y recuperación 22

18. Mantenimiento y solución de problemas 22

19. Limitaciones y evolución técnica 23

20. Requerimientos mínimos del sistema 24

Referencias 26

Apéndices [PÁGINA]

1. Propósito y alcance técnico

El manual proporciona la información necesaria para instalar, configurar, ejecutar, validar y mantener la primera entrega de Diplomatic Fleet en un entorno local. Está dirigido al equipo de desarrollo, personal evaluador y personas responsables de preparar la aplicación en otro equipo.

1.1 Alcance implementado

- Frontend Vue.js con TypeScript, Vue Router, servicios HTTP y diseño responsive.
- Backend Kotlin con Spring Boot, controladores REST, servicios, repositorios JPA, DTO y excepciones.
- Persistencia PostgreSQL para Clientes institucionales y Vehículos.
- Migraciones Flyway para crear y versionar el esquema.
- Spring Security con HTTP Basic temporal para la demo.
- Navegación común para diez módulos.
- Compilación y pruebas mediante Gradle Wrapper y npm.

1.2 Alcance pendiente

Documentos, Mantenimientos, Disponibilidad, Solicitudes, Validación de conductor, Cotizaciones y Usuarios y roles disponen de interfaz navegable, pero requieren completar entidades, reglas de negocio, endpoints, persistencia y pruebas. La autenticación definitiva basada en usuarios y roles también permanece pendiente.

2. Descripción general del sistema

Diplomatic Fleet centraliza operaciones vehiculares destinadas a clientes institucionales. El navegador ejecuta una aplicación Vue que consume rutas /api. Durante el desarrollo, Vite redirige estas rutas al backend en el puerto 8080. Spring Boot procesa las solicitudes, valida

reglas y utiliza JPA para acceder a PostgreSQL. La base de datos escucha habitualmente en el puerto 5432.

Componente	Responsabilidad
Navegador	Ejecutar la interfaz y mantener la sesión temporal.
Vue.js	Renderizar vistas, formularios, indicadores y navegación.
Servicios TypeScript	Construir solicitudes, autenticación y mensajes de error.
Spring Security	Proteger la API con HTTP Basic temporal.
Controladores REST	Exponer endpoints y devolver códigos HTTP.
Servicios Kotlin	Aplicar reglas, transacciones y normalización.
Repositorios JPA	Abstraer consultas y persistencia.
Flyway	Aplicar migraciones en orden.
PostgreSQL	Conservar los datos del sistema.

3. Arquitectura de solución

La arquitectura adopta separación de responsabilidades. El frontend no accede directamente a PostgreSQL. Toda operación persistente atraviesa la API, los servicios de negocio y los repositorios. Esta decisión permite centralizar validaciones y mantener una fuente única de verdad.

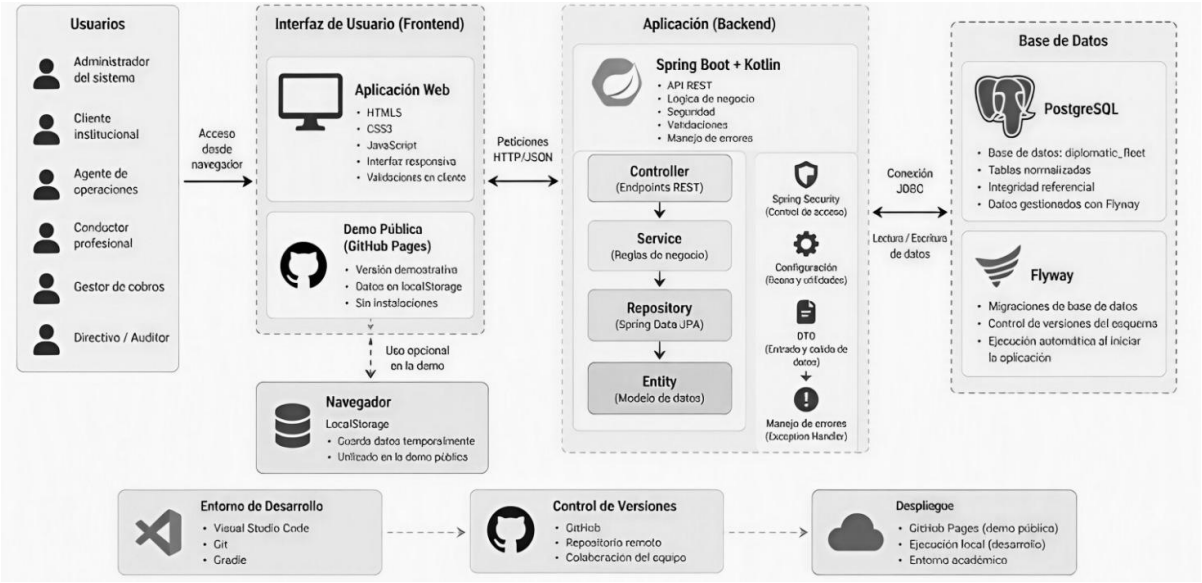


Figura 1. Arquitectura general de Diplomatic Fleet.

Nota. Elaboración propia del equipo. Imagen convertida a escala de grises para la entrega académica.

3.1 Componentes

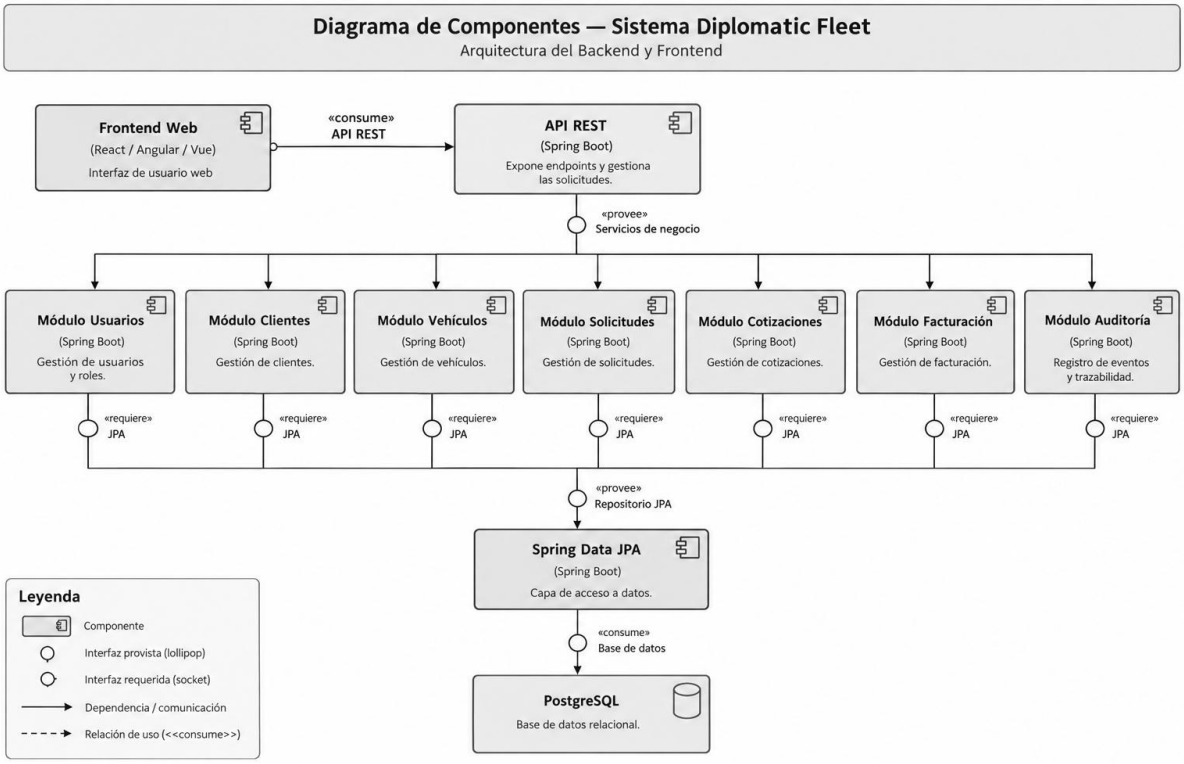


Figura 2. Diagrama de componentes.

Nota. Elaboración propia del equipo. Se representan los componentes técnicos y sus dependencias.

3.2 Paquetes

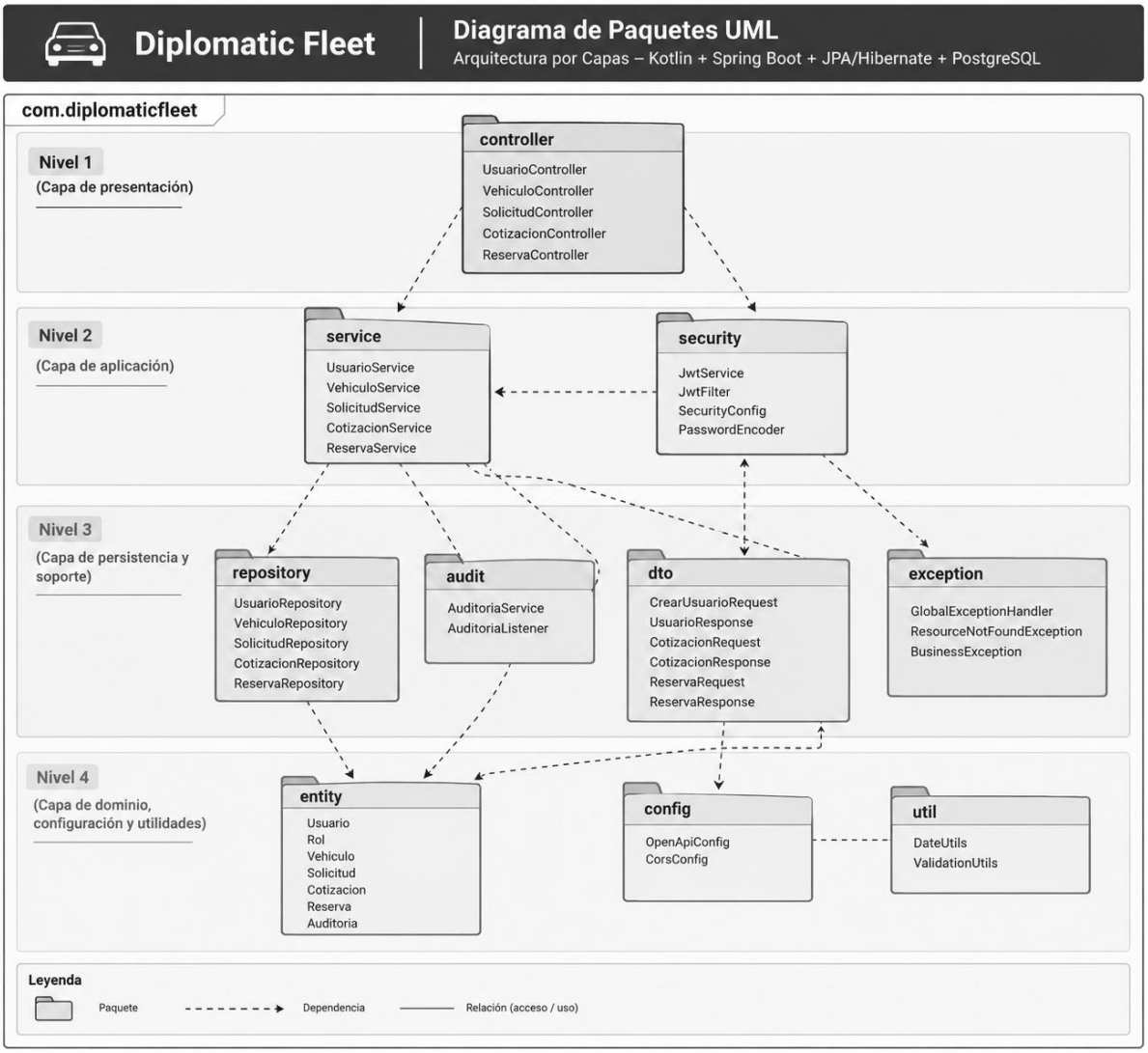


Figura 3. Diagrama de paquetes.

Nota. Elaboración propia del equipo. La estructura modular separa dominio, persistencia, servicios y presentación.

3.3 Modelo de dominio

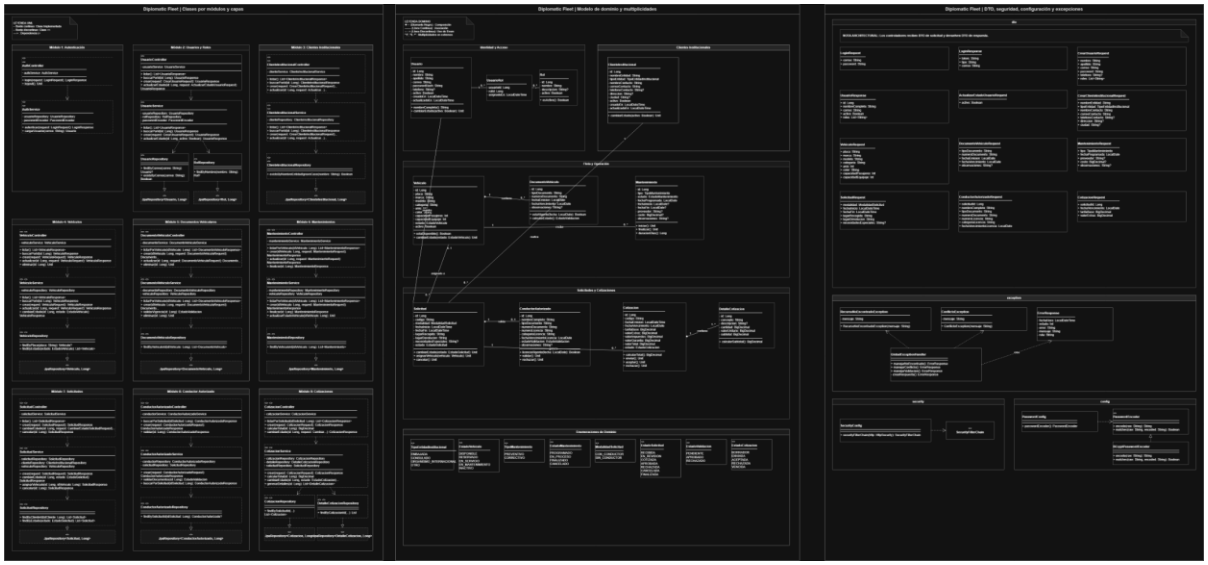


Figura 4. Modelo de dominio UML.

Nota. Elaboración propia del equipo. Clientes y Vehículos son los módulos funcionales de la primera entrega.

3.4 Secuencia principal

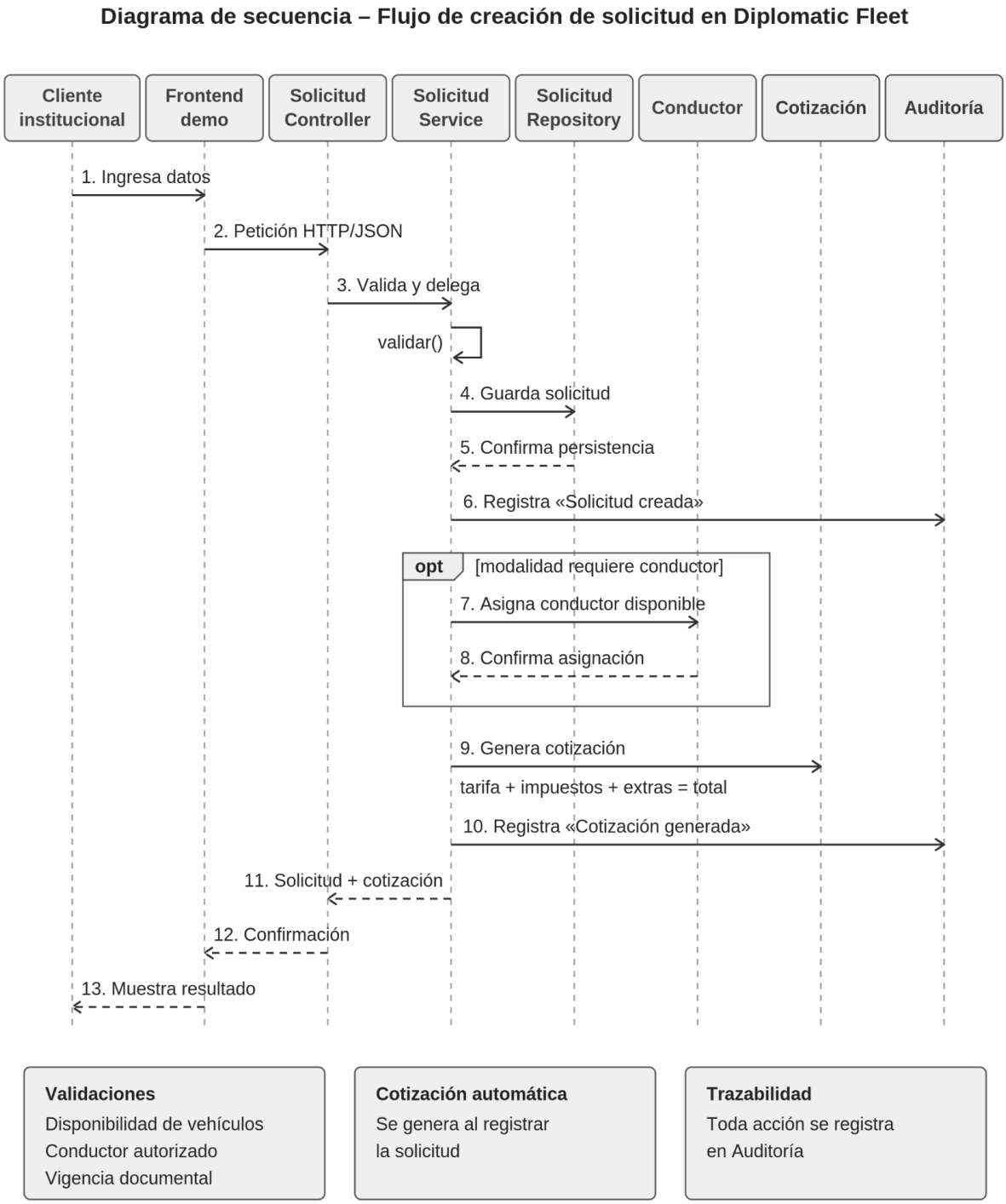


Figura 5. Secuencia del flujo principal.

Nota. Elaboración propia del equipo. La secuencia completa constituye el objetivo de evolución del sistema.

3.5 Estados

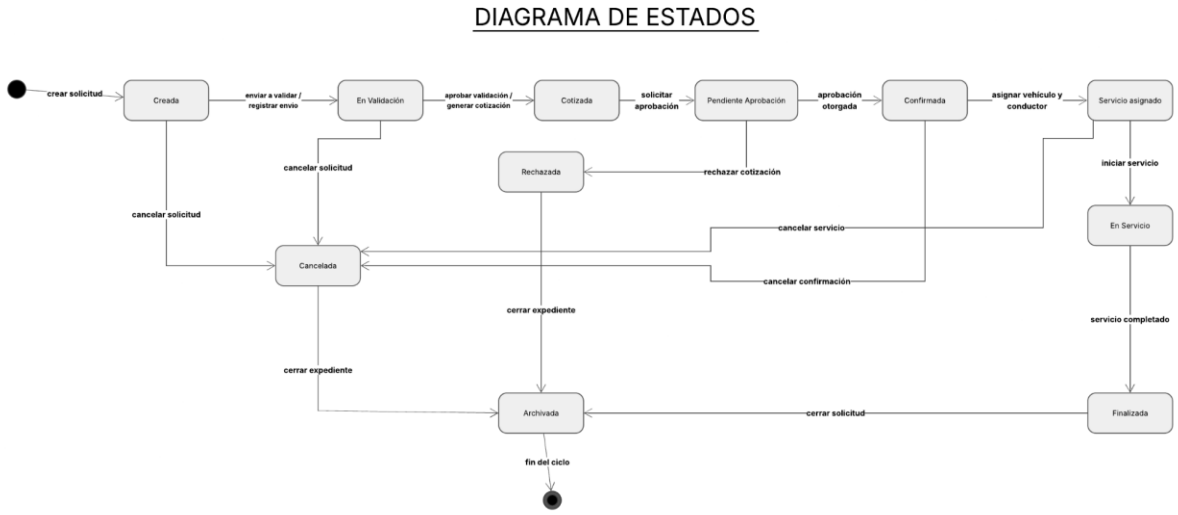


Figura 6. Estados del dominio.

Nota. Elaboración propia del equipo. Los estados avanzados requieren integración con reservas, servicios y mantenimiento.

4. Estructura del código fuente

Ruta	Contenido
src/main/kotlin/com/diplomaticfleet/backend	Código Kotlin del backend.
src/main/resources	Configuración, recursos y migraciones.
src/main/resources/db/migration	Scripts Flyway versionados.
src/test	Pruebas de Spring Boot.
frontend/src/components	Componentes visuales reutilizables.
frontend/src/components/layout	DemoSidebar y DemoLayout.
frontend/src/services	vehiculoService.ts y clienteService.ts.
frontend/src/views	Login, Clientes, Vehículos y vistas demo.
frontend/src/router	Configuración de rutas.
frontend/src/assets	Estilos e imágenes.
docs	Documentación técnica del repositorio.

4.1 Patrón de backend

Capa	Responsabilidad	Ejemplos

Dominio	Mapeo del estado persistente.	ClienteInstitucional, Vehiculo, TipoEntidad.
Repository	Operaciones JPA y consultas derivadas.	ClienteInstitucionalRepository.
DTO	Contrato de entrada y salida.	ClienteInstitucionalRequest y Response.
Service	Reglas, normalización y transacciones.	ClienteInstitucionalService.
Controller	Rutas HTTP y códigos de respuesta.	ClienteInstitucionalController.
Exception	Tratamiento uniforme de errores.	GlobalExceptionHandler.

4.2 Patrón de frontend

Cada módulo funcional utiliza una vista Vue y un servicio TypeScript. La vista administra el estado reactivo, formularios e indicadores; el servicio construye fetch, cabeceras, autenticación y traducción de errores. DemoSidebar y DemoLayout evitan duplicar la navegación en cada módulo.

5. Tecnologías y dependencias

Tecnología	Uso
Kotlin	Lenguaje del backend.
Spring Boot	Inicialización, servidor y configuración.
Spring Web	Controladores REST y JSON.
Spring Data JPA / Hibernate	Persistencia orientada a entidades.
Spring Validation	Validación de DTO mediante anotaciones.
Spring Security	Protección temporal con HTTP Basic.
Flyway	Migraciones del esquema.
PostgreSQL	Base de datos relacional.
Gradle Wrapper	Construcción reproducible del backend.
Vue.js	Interfaz de usuario reactiva.
TypeScript	Tipado estático del frontend.
Vue Router	Navegación SPA.
Vite	Servidor de desarrollo, proxy y build.

npm	Gestión de dependencias del frontend.
Git y GitHub	Versionamiento, ramas y Pull Requests.

5.1 Reglas de compatibilidad

El backend se ejecuta con Java JDK 21. El frontend declara compatibilidad de Node.js en package.json y debe respetarse al instalar el proyecto. Gradle se ejecuta mediante gradlew o gradlew.bat, por lo que no se requiere una instalación global de Gradle.

6. Configuración del backend

6.1 Variables de entorno

Variable	Ejemplo no sensible	Finalidad
DB_URL	jdbc:postgresql://localhost:5432/diplomatic_fleet	URL JDBC de PostgreSQL.
DB_USERNAME	postgres	Usuario local de base de datos.
DB_PASSWORD	No documentar	Contraseña local de PostgreSQL.
SPRING_SECURITY_USER_NAME	user	Usuario técnico temporal.
SPRING_SECURITY_USER_PASSWORD	Demo1234	Contraseña técnica de la demo.

Las variables se configuran en la terminal que iniciará Spring Boot. Al cerrar la terminal se pierden. La contraseña real de PostgreSQL no debe incluirse en Git, documentación pública, capturas ni mensajes compartidos.

6.2 Configuración en PowerShell

```
$env:DB_URL="jdbc:postgresql://localhost:5432/diplomatic_fleet"
$env:DB_USERNAME="postgres"
$env:DB_PASSWORD="CONTRASEÑA_LOCAL"
$env:SPRING_SECURITY_USER_NAME="user"
$env:SPRING_SECURITY_USER_PASSWORD="Demo1234"
.\gradlew.bat bootRun
```

6.3 Perfil de datos

application.properties obtiene la conexión desde variables de entorno. Hibernate valida el esquema administrado por Flyway, evitando que el ORM modifique automáticamente las tablas. Si el esquema no coincide con las entidades, el inicio debe fallar para impedir una ejecución inconsistente.

7. Configuración del frontend

7.1 Instalación de dependencias

```
cd frontend
npm install
npm run build
```

node_modules no se copia entre equipos ni se versiona. npm install reconstruye las dependencias desde package.json y el archivo de bloqueo del proyecto.

7.2 Desarrollo con Vite

```
cd frontend
npm run dev
```

Vite muestra la dirección local del frontend. vite.config.ts contiene un proxy que redirige /api al backend de Spring Boot. El backend debe estar activo antes de validar el login o consultar datos.

7.3 Compilación de producción

npm run build ejecuta la verificación de TypeScript y la generación de archivos optimizados. La carpeta dist es un resultado de compilación y no forma parte de los archivos fuente que se confirman en Git.

7.4 Sesión del navegador

Las credenciales técnicas se almacenan temporalmente en sessionStorage. clienteService.ts y vehiculoService.ts las utilizan para construir Authorization: Basic y X-

Requested-With: XMLHttpRequest. El segundo encabezado permite que los errores 401 se manejen dentro de la interfaz.

8. Base de datos y migraciones

8.1 Creación inicial

```
CREATE DATABASE diplomatic_fleet;
```

La base de datos debe existir antes de iniciar Spring Boot. Flyway crea las tablas y demás objetos definidos en las migraciones, pero no crea el servidor PostgreSQL ni la base contenedora.

8.2 Convención de migraciones

Las migraciones se ubican en src/main/resources/db/migration y utilizan una versión ascendente. Una migración aplicada no debe editarse en un entorno compartido; las modificaciones posteriores deben incorporarse en una nueva versión.

8.3 Integridad

- Claves primarias generadas por identidad.
- Claves foráneas para relaciones entre entidades.
- Restricciones CHECK para valores admitidos.
- Índices únicos para nombres, correos, identificaciones y placas.
- Fechas de creación y actualización.
- Borrado lógico mediante campos activo cuando corresponde.

8.4 Respaldo y traslado

Git transfiere el código y las migraciones, pero no los registros locales. Para mover datos entre equipos debe generarse y restaurarse un respaldo de PostgreSQL autorizado, o registrar nuevamente datos ficticios de presentación.

9. Módulo de clientes institucionales

Elemento	Implementación
Entidad	ClienteInstitucional mapea clientes_institucionales.
Enum	TipoEntidad: EMBAJADA, CONSULADO, ORGANISMO_INTERNACIONAL y OTRO.
Repositorio	Consultas por nombre, correo, identificación, estado y tipo.
Servicio	Registro, listado, consulta, actualización y control de unicidad.
Controlador	Ruta base /api/clientes.
Frontend	clienteService.ts y ClientesView.vue.

9.1 Endpoints

Método	Ruta	Respuesta principal
POST	/api/clientes	201 Created.
GET	/api/clientes	200 OK con lista.
GET	/api/clientes/{id}	200 OK o 404.
PUT	/api/clientes/{id}	200 OK, 404 o 409.

9.2 Reglas técnicas

- Nombre, contacto y correo son obligatorios.
- El correo debe tener formato válido.
- Nombre de entidad y correo son únicos sin distinguir mayúsculas.
- La identificación fiscal es única cuando se suministra.
- Los opcionales vacíos se convierten en null.
- El correo se normaliza a minúsculas.
- La identificación fiscal se normaliza a mayúsculas.
- La inactivación conserva el registro.

10. Módulo de vehículos

Elemento	Implementación
Dominio	Vehículo, categoría, estados y combustible.
Repositorio	Consulta y persistencia JPA.
Servicio	Validación de categoría, normalización de placa y duplicados.
Controlador	Ruta base /api/vehiculos.
Frontend	vehiculoService.ts y VehiculosView.vue.

10.1 Endpoints

Método	Ruta	Respuesta principal
POST	/api/vehiculos	201 Created.
GET	/api/vehiculos	200 OK con lista.
GET	/api/vehiculos/{id}	200 OK o 404.
PUT	/api/vehiculos/{id}	200 OK, 404 o 409.

10.2 Reglas técnicas

- La placa se normaliza a mayúsculas y debe ser única.
- La categoría debe existir y estar activa.
- Los textos se recortan antes de persistir.
- El año y las capacidades se validan en el contrato.
- El listado alimenta los indicadores del frontend.
- Las transiciones de reserva, servicio y mantenimiento requieren completar los módulos relacionados.

11. Seguridad y autenticación temporal

La primera entrega utiliza HTTP Basic con un usuario técnico configurado mediante variables de entorno. El formulario exige un correo fijo de demostración y valida la contraseña

llamando a un endpoint protegido. Esta arquitectura permite una demo controlada, pero no sustituye un sistema de autenticación productivo.

Aspecto	Estado
Protección de /api	Activa mediante Spring Security.
Usuario real de PostgreSQL	No utilizado para autenticación de personas usuarias.
Correo fijo	Temporal para la demo.
sessionStorage	Contiene credenciales técnicas durante la sesión.
HTTPS	No configurado en ejecución local.
Roles y permisos	Pendientes de integración.
BCrypt	Previsto para cuentas reales.

11.1 Recomendaciones para producción

- Eliminar las credenciales fijas del frontend.
- Implementar cuentas reales con contraseñas codificadas.
- Aplicar autorización por rol a endpoints y vistas.
- Utilizar HTTPS.
- Definir expiración y renovación de sesión.
- Proteger secretos mediante un gestor o variables del entorno de despliegue.
- Habilitar auditoría y monitoreo.

12. Manejo de errores y validaciones

Código	Origen típico	Tratamiento
400	DTO inválido o dato obligatorio ausente.	GlobalExceptionHandler devuelve validaciones.
401	Credenciales inválidas.	Frontend elimina sesión y muestra mensaje.
404	Entidad inexistente.	RecursoNoEncontradoException.
409	Nombre, correo, identificación o placa duplicados.	ConflictoException.

500	Fallo no controlado.	Registrar diagnóstico sin exponer secretos.

La regla técnica consiste en mantener las validaciones decisivas en el backend. El frontend mejora la experiencia con atributos HTML y mensajes, pero no reemplaza la protección del servicio.

13. Instalación desde cero

1. Instalar Git, JDK 21, Node.js compatible y PostgreSQL.
2. Clonar el repositorio desde GitHub.
3. Cambiar a main y ejecutar git pull origin main.
4. Crear la base diplomatic_fleet.
5. Entrar en frontend y ejecutar npm install.
6. Configurar las variables del backend en PowerShell.
7. Ejecutar .\gradlew.bat test.
8. Ejecutar npm run build dentro de frontend.
9. Iniciar el backend con bootRun.
10. Iniciar el frontend con npm run dev en otra terminal.

13.1 Clonado

```
git clone https://github.com/Riiperk/Diplomatic-Fleet.git
cd Diplomatic-Fleet
git switch main
git pull origin main
git status
```

13.2 Criterios de instalación correcta

- Git informa que main está actualizada.
- Gradle informa BUILD SUCCESSFUL.
- Vite informa built o muestra la URL local.
- Spring Boot inicia Tomcat en 8080.

• El login acepta las credenciales de demo.

• Clientes y Vehículos consultan PostgreSQL.

14. Ejecución y apagado controlado

14.1 Orden de inicio

1. Iniciar PostgreSQL.
2. Abrir la terminal del backend y configurar variables.
3. Ejecutar bootRun y esperar el inicio de Tomcat.
4. Abrir otra terminal en frontend y ejecutar npm run dev.
5. Abrir el navegador y probar el login.

14.2 Orden de apagado

1. Cerrar sesión en la aplicación.
2. Detener Vite con Ctrl + C.
3. Detener Spring Boot con Ctrl + C.
4. Cerrar las terminales.
5. Detener PostgreSQL solo si la política del equipo lo requiere.

La interrupción con Ctrl + C permite que HikariCP y JPA cierren los recursos de forma ordenada. No se debe finalizar la tarea de Java durante una escritura salvo que el proceso no responda.

15. Pruebas y aseguramiento de calidad

Comando o prueba	Objetivo	Resultado esperado
.\gradlew.bat compileKotlin	Compilar código Kotlin.	BUILD SUCCESSFUL.
.\gradlew.bat test	Cargar contexto y ejecutar pruebas.	BUILD SUCCESSFUL.
npm run build	Validar TypeScript y compilar Vue.	built.
Login erróneo	Validar rechazo.	Permanece en login.

Login correcto	Validar acceso.	Navega a /inicio.
Registrar cliente	Validar POST y persistencia.	Aparece y permanece.
Cambiar estado	Validar PUT.	Estado persiste.
Registrar vehículo	Validar POST.	Aparece en listado.
Actualizar listados	Validar GET.	Recupera PostgreSQL.

15.1 Cobertura pendiente

- Pruebas unitarias de todos los servicios.
- Pruebas MockMvc de controladores.
- Pruebas de repositorio con base aislada.
- Casos de concurrencia y duplicados.
- Pruebas de autorización por rol.
- Pruebas extremo a extremo.
- Pruebas de accesibilidad y rendimiento.

16. Control de versiones y despliegue

El flujo utilizado parte de main actualizada, crea una rama feature, realiza commits por bloque, publica la rama, crea un Pull Request, revisa Files changed desde otra cuenta, aprueba y fusiona. Después del merge se actualiza main local y se ejecutan las pruebas finales.

Elemento	Convención utilizada
Ramas	feature/modulo-responsable.
Commits	Conventional Commits: feat, docs, style.
Autoría	user.name y user.email locales del responsable.
Revisión	Comentarios técnicos en Files changed.
Integración	Pull Request hacia main.
Artefactos	dist, node_modules y secretos excluidos.

16.1 Despliegue actual

La primera entrega se ejecuta localmente. Para un despliegue público completo se requiere alojar el backend, proporcionar una instancia PostgreSQL, configurar variables del entorno remoto, definir HTTPS, CORS, registro de logs y una estrategia de respaldo. El frontend está preparado para compilarse, pero su URL de API debe corresponder al entorno publicado.

17. Copia, migración y recuperación

17.1 Traslado sin datos

El método recomendado es clonar main y permitir que Flyway cree el esquema en una base nueva. Este método transfiere código, historia y migraciones, pero no los registros.

17.2 Traslado con datos

Cuando se requiera conservar registros, debe utilizarse una herramienta oficial de respaldo y restauración de PostgreSQL. El archivo de respaldo debe protegerse porque puede contener datos institucionales. Para la presentación académica es suficiente registrar nuevamente datos ficticios.

17.3 Recuperación de código

- Confirmar git remote -v.
- Ejecutar git fetch origin.
- Cambiar a main.
- Ejecutar git pull origin main.
- Verificar git status.
- No sobrescribir cambios locales sin crear una copia o commit.

18. Mantenimiento y solución de problemas

Problema	Diagnóstico	Solución

Puerto 8080 ocupado	Otro proceso usa el backend.	Cerrar el proceso o cambiar el puerto de forma controlada.
Puerto 5173 ocupado	Otra instancia de Vite.	Detenerla o usar la URL alternativa mostrada.
Connection refused 5432	PostgreSQL detenido o puerto diferente.	Iniciar el servicio y revisar DB_URL.
Password authentication failed	DB_PASSWORD incorrecta.	Configurar la contraseña local correcta.
Flyway checksum mismatch	Se editó una migración aplicada.	Restaurar la migración y crear una nueva versión.
Schema validation failed	Entidad y tabla no coinciden.	Revisar @Column, nulabilidad y migraciones.
401 en API	Credenciales técnicas distintas.	Alinear variables y volver a iniciar sesión.
Cuadro nativo Basic Auth	Falta X-Requested-With.	Verificar los servicios TypeScript.
Error de Vue	Etiqueta o TypeScript inválido.	Ejecutar npm run build y corregir línea indicada.
Datos no aparecen en otra laptop	La base es local.	Restaurar respaldo o crear datos de demo.

18.1 Mantenimiento preventivo

- Ejecutar pruebas antes y después de cada integración.
- Mantener dependencias y versiones documentadas.
- No editar migraciones aplicadas.
- Revisar logs sin publicar secretos.
- Comprobar respaldos antes de cambios de esquema.
- Eliminar datos de prueba inadecuados antes de presentar.
- Actualizar documentación y diagramas cuando cambie la arquitectura.

19. Limitaciones y evolución técnica

- HTTP Basic y correo fijo son soluciones temporales.
- No existe despliegue público completo del backend.

• Los módulos demostrativos carecen de API y persistencia.

• Las pruebas automatizadas requieren ampliación.

• La edición completa de clientes aún no está disponible en Vue.

• La disponibilidad no cruza todavía rangos, reservas y mantenimientos.

• No existe telemetría, auditoría completa ni estrategia formal de recuperación ante desastres.

La prioridad recomendada es implementar usuarios y roles reales, después integrar Solicitudes y Disponibilidad, y a continuación Documentos, Mantenimientos y Cotizaciones. Cada integración debe repetir el patrón de entidad, repositorio, DTO, servicio, controlador, servicio TypeScript, vista y pruebas.

20. Requerimientos mínimos del sistema

Esta sección se presenta al final del manual técnico como requisito de la entrega. Los valores corresponden a una ejecución local académica y no constituyen una capacidad dimensionada para producción.

20.1 Hardware mínimo

Recurso	Mínimo	Recomendado
Procesador	Dos núcleos a 2 GHz o equivalente.	Cuatro núcleos o superior.
Memoria RAM	4 GB.	8 GB o más.
Almacenamiento	1 GB libre más espacio de PostgreSQL.	5 GB libres para herramientas, dependencias y respaldos.
Pantalla	1366 × 768.	1920 × 1080.
Conectividad	Internet durante instalación inicial.	Conexión estable para GitHub y actualizaciones.

20.2 Software mínimo

Componente	Versión o condición

Sistema operativo	Windows 10/11 de 64 bits, Linux o macOS compatible.
Java	JDK 21.
Node.js	^22.18.0 o >=24.12.0, según package.json.
npm	Incluido con Node.js.
PostgreSQL	Servidor compatible con las migraciones del proyecto.
Git	Cliente disponible en terminal.
Navegador	Edge, Chrome o Firefox actualizado.
IDE opcional	IntelliJ IDEA o Visual Studio Code.

20.3 Puertos y servicios

Puerto	Servicio	Condición
5432	PostgreSQL	Disponible y accesible desde Spring Boot.
8080	Spring Boot	Libre para el backend.
5173	Vite	Libre o aceptar el puerto alternativo mostrado.

20.4 Requerimientos de configuración

- Base diplomatic_fleet creada.
- Migraciones disponibles en el repositorio.
- Variables de entorno configuradas en la terminal del backend.
- Dependencias instaladas mediante npm install.
- Rama main actualizada y árbol de trabajo limpio.
- Credenciales de demostración preparadas.
- Datos ficticios de presentación registrados.

20.5 Criterios de aceptación técnica

Verificación	Resultado esperado
git status	Main actualizada y sin cambios pendientes.
.\gradlew.bat test	BUILD SUCCESSFUL.

npm run build	TypeScript y Vite finalizan sin errores.
bootRun	Tomcat inicia en 8080.
npm run dev	Frontend abre en navegador.
Login	Rechaza credenciales erróneas y acepta la demo.
Clientes	Registro, búsqueda y cambio de estado persistentes.
Vehículos	Registro, búsqueda y consulta persistentes.

Referencias

American Psychological Association. (2025). Student paper setup guide (7th ed.). https://apastyle.apa.org/instructional-aids/student-paper-setup-guide.pdf

Equipo Diplomatic Fleet. (2026). Documentación técnica y funcional de la primera entrega [Documento académico no publicado].

Git. (2026). Git clone documentation. https://git-scm.com/docs/git-clone

Node.js. (2026). Download Node.js. https://nodejs.org/en/download

PostgreSQL Global Development Group. (2026). PostgreSQL documentation. https://www.postgresql.org/docs/

Spring. (2026). Spring Boot reference documentation. https://docs.spring.io/spring-boot/reference/

Vue.js. (2026). Vue.js documentation. https://vuejs.org/guide/

Apéndice A. Comandos operativos

```
git switch main
git pull origin main
git status

.\gradlew.bat test
.\gradlew.bat bootRun

cd frontend
npm install
npm run build
npm run dev
```

Apéndice B. Endpoints funcionales

Método	Ruta
POST	/api/clientes
GET	/api/clientes
GET	/api/clientes/{id}
PUT	/api/clientes/{id}
POST	/api/vehiculos
GET	/api/vehiculos
GET	/api/vehiculos/{id}
PUT	/api/vehiculos/{id}